Sample 3: Introduction to Document Processing for RAG

Chapter 1 — What Is Document Ingestion?

Document ingestion is the process of bringing source content into an application so it can be searched, analyzed, or used by downstream systems. In a retrieval-augmented generation pipeline, ingestion commonly includes loading, normalization, splitting, metadata assignment, embedding, and indexing.

Why Source Quality Matters

Text extraction quality affects every later stage. A scanned PDF may contain page images rather than selectable text, which means an ordinary text loader may return little or no useful content unless optical character recognition is used. Multi-column layouts, headers, footers, and tables can also produce text in an unexpected order.

Chapter 2 — Loading Documents

A document loader converts a source file into one or more Document objects. In LangChain, a Document commonly contains page_content and metadata. The page_content field stores extracted text, while metadata may include a source path, page index, title, or other source-specific details.

Choosing a PDF Loader

Different PDF loaders use different extraction libraries and may behave differently on complex layouts. Test candidate loaders against representative files rather than assuming that one loader is best for every document. Compare extracted text, page boundaries, table readability, and processing time.

A Basic Validation Checklist

• Confirm the file exists and can be opened.
• Check the number of pages returned by the loader.
• Inspect text from the first, middle, and last pages.
• Verify metadata includes the source and page information where expected.
• Flag empty pages or unusually short extracted text for review.

Chapter 3 — Splitting Documents

Chunking divides long documents into smaller pieces. Embedding models and retrieval systems work with bounded input sizes, and focused chunks can improve the relevance of retrieved passages. However, chunks that are too small may lose context, while chunks that are too large may combine several unrelated topics.

Chunk Size and Overlap

Chunk size is often configured in characters for a character-based splitter, although embedding models ultimately process tokens. Overlap repeats part of one chunk in the next. Overlap can help preserve facts that cross a boundary, but excessive overlap increases storage and embedding costs and may create near-duplicate search results.

Recursive Splitting

RecursiveCharacterTextSplitter tries a sequence of separators, commonly paragraph breaks, line breaks, spaces, and finally individual characters. It attempts to keep meaningful units together while respecting the target chunk size. The exact outcome depends on the input text, separators, chunk size, and overlap.

Chapter 4 — Metadata and Evaluation

Metadata helps filter and interpret retrieved chunks. Common fields include source, page number, document title, section heading, tenant identifier, document type, and publication date. Metadata should be accurate and should not contain secrets that do not belong in the retrieval index.

A Simple Experiment

Use the same source document to create three chunk sets: 200 characters with 30 characters of overlap, 500 characters with 75 characters of overlap, and 1,000 characters with 150 characters of overlap. Count the chunks, inspect several boundaries, and ask the same factual questions against each set.

Questions for Evaluation

1. Which chunk size keeps a heading and its paragraph together most often?
2. Does overlap preserve information that would otherwise be split across chunks?
3. Are retrieved passages focused enough to answer a question without unnecessary surrounding text?
4. Do page and source metadata remain available after splitting?

Chapter 5 — Practical Notes

Do not assume that a successful import proves that the correct environment is active. When a package cannot be imported, inspect the Python executable used by the notebook kernel and compare it with the project environment. Install dependencies into the environment that actually runs the code, then restart the kernel if needed.

End of sample document.